CNN BASED CHEST RADIOGRAPH CLASSIFIER FOR COVID-19 IDENTIFICATION

Team Members:

MEGHANA P B

SUKHI R

T R SNEHA

USN:

1CR22AD065

1CR22AD110

1CR22AD114

Department: ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Institution: CMR INSTITUTE OF TECHNOLOGY

Batch: 2022 – 2026

Abstract

The COVID-19 pandemic has exerted tremendous pressure on healthcare systems across the globe, underscoring the urgent need for rapid and reliable diagnostic techniques. Among the available diagnostic modalities, chest X-ray imaging remains one of the most cost-effective, widely accessible, and efficient methods for detecting respiratory illnesses such as COVID-19. However, manual examination of X-ray images by radiologists is often time-intensive, subjective, and susceptible to variations in interpretation, especially during mass screening scenarios in pandemic conditions.

To address these challenges, this project proposes an automated COVID-19 detection system leveraging deep learning techniques, particularly **Convolutional Neural Networks (CNNs)**. Pre-trained CNN architectures such as **ResNet50, VGG16, and Xception** were employed using transfer learning to **classify chest X-ray images into two categories — COVID and Non-COVID.** By adopting transfer learning, the models achieved high levels of accuracy, robustness, and generalization even when trained on limited medical imaging data, thereby ensuring both efficiency and reliability in real-world applications..

To enhance usability, the trained models were integrated into a **Django-based web application** that allows users to upload chest X-ray images and instantly receive diagnostic predictions. The web interface is designed to be intuitive, ensuring that healthcare professionals and researchers can easily interpret the results without requiring technical expertise.

The proposed system provides a **scalable, reliable, and fast diagnostic aid**, helping reduce the workload of medical experts while improving diagnostic consistency. This project demonstrates the potential of **AI-driven solutions** in supporting radiological assessments and contributing to public health efforts, particularly in managing large-scale health emergencies like the COVID-19 pandemic.

Table of Contents

1 Abstract

2 Introduction

3 Problem Definition

4 Motivation And Related Works

5 Objective Of The Project

6 Existing System & Proposed System

7 System Architecture

7.1 Workflow Overview

8 Modules Description

9 Algorithms and Models Used

10 Mathematical Model

11 Dataset Description

11.1 Overview of the Dataset

11.2 Dataset Composition

11.3 Data Preprocessing and Augmentation

11.4 Dataset Splitting

11.5 Significance of the Dataset

12 System Requirements

13 Implementation Details

14 Results,Interface and System Evaluation

14.1 Screenshots and User Interface Description

14.2 Results and Discussion

14.3 Advantages and Applications

14.4 Limitations and Future Enhancement

14.5 Conclusion

2. Introduction

The **COVID-19 pandemic**, caused by the SARS-CoV-2 virus, has severely affected global health and economies. With millions of infections worldwide, there is an urgent need for **fast and accurate diagnostic tools** to contain the virus and improve patient outcomes. Although **RT-PCR** remains the gold standard for COVID-19 detection, it is **time-consuming, expensive,** and not always available in remote areas, creating a need for alternative solutions.

Chest X-ray imaging has become a practical and affordable method for detecting **respiratory infections**, including COVID-19. It provides quick insight into lung conditions, but **manual interpretation by radiologists** can be slow, subjective, and prone to errors, especially during large-scale screenings. These challenges emphasize the need for **AI-based automation** to enhance diagnostic speed and accuracy.

This project develops an **AI-driven web system** for detecting COVID-19 from chest X-rays using **Convolutional Neural Networks (CNNs)**. Pre-trained models like **ResNet50, VGG16,** and **Xception** are applied through **transfer learning** to classify X-rays into **COVID** and **Non-COVID** categories. This approach improves **accuracy, reliability,** and performance, even with limited data.

The proposed system integrates these trained CNN models into a **Django-based web application**, providing users with an intuitive interface to upload chest X-ray images and receive real-time predictions. This integration bridges the gap between **AI research and practical healthcare deployment** enabling radiologists and clinicians to make faster and more consistent diagnoses. The system not only reduces human workload but also contributes to early detection and treatment, which is crucial in controlling the spread of infectious diseases. Overall, this project demonstrates how **AI-powered diagnostic tools** can play a transformative role in modern healthcare, offering a scalable and efficient solution for disease identification and management.

3. Problem Definition

The **COVID-19 pandemic** has exposed several weaknesses in global healthcare systems, especially during times of **rapid case surges** and **resource shortages**. Hospitals and laboratories faced immense pressure to diagnose patients quickly while managing limited staff and testing capacity.

Traditional diagnostic methods, such as **RT-PCR tests**, though highly accurate, come with notable limitations:

1. **Time-Consuming Process** – RT-PCR testing involves multiple stages, including sample collection, RNA extraction, amplification, and result analysis. This leads to significant delays, especially when testing demand is high.
2. **High Cost and Limited Availability** – The testing kits, reagents, and necessary laboratory infrastructure are costly and often not available in sufficient quantities during peak outbreaks.
3. **Dependence on Skilled Personnel** – The process requires **trained laboratory technicians** and **specialized equipment**, which are not always accessible in **rural or under-resourced regions**, making testing coverage uneven.

As a result of these challenges, **manual diagnostic methods** become inefficient and unsustainable during pandemic conditions. Radiologists interpreting chest X-ray images manually also face limitations such as **work overload, fatigue, and diagnostic variability**. Human judgment can vary across individuals, leading to **inconsistencies in interpretation** and an increased risk of misdiagnosis when handling thousands of cases daily.

Hence, there arises a **pressing need for an automated, reliable, and scalable diagnostic system** capable of providing **rapid and consistent results**. Leveraging **chest X-ray imaging**, which is readily available in most healthcare facilities, offers a practical solution. By integrating **artificial intelligence (AI)** and **deep learning models** into this process, it becomes possible to significantly **reduce diagnostic time**, **enhance accuracy**, and **assist medical professionals** in making well-informed decisions under time-critical conditions.

4. Motivation & Related Works

To address the above challenges, this project is driven by the **motivation to develop an intelligent, AI-powered detection system** that can assist in rapid and accurate COVID-19 diagnosis. The integration of **deep learning techniques**, particularly **Convolutional Neural Networks (CNNs)**, offers a promising solution for medical image analysis.

The proposed system aims to:

1) **Automate the detection process** using CNN-based models trained on chest X-ray datasets.
2) **Minimize human error** and **enhance diagnostic consistency** through AI-driven analysis.
3) **Provide real-time predictions** integrated into a web-based interface using Django.
4) **Improve accessibility** of diagnostic tools to remote or resource-limited regions.
5) **Reduce the workload** of healthcare professionals, allowing them to focus on treatment rather than manual image evaluation.

Such **AI-driven automation** bridges the gap between advanced computational intelligence and practical healthcare applications. It not only ensures **faster decision-making** and **cost-effective screening** but also contributes to **early detection and control** of infectious diseases during global health crises.

Ultimately, this project is motivated by the goal of building a **scalable, efficient, and clinically useful diagnostic system** that can support radiologists and clinicians in saving time, improving accuracy, and enhancing patient care outcomes.

Researchers have explored various deep learning architectures for COVID-19 detection using chest X-rays. **ResNet-based models** (Narin et al., 2020) effectively handled vanishing gradients and achieved high accuracy in distinguishing COVID-19 from normal and pneumonia cases. **VGG models** (Apostolopoulos & Mpesiana, 2020) offered strong performance when fine-tuned, showing high sensitivity and specificity. **Xception and Inception architectures** improved efficiency through depthwise separable convolutions, enhancing feature extraction and reducing computation.

To boost reliability, **ensemble and hybrid approaches** combined models like ResNet, VGG, and Xception, achieving better generalization and lower classification errors. However, many studies emphasized accuracy over usability. Thus, recent efforts focus on **web-based implementations** using frameworks like Django or Flask to enable real-time, user-friendly access.

This project extends prior work by integrating multiple pre-trained CNNs with ensemble learning and deploying them through a Django-based interface for practical, real-world application.

5. Objectives of the Project

The primary goal of this project is to design and implement an **AI-powered web application** capable of detecting COVID-19 infection from **chest X-ray images** using **Convolutional Neural Network (CNN)** architectures. The project focuses on combining the power of deep learning models with an intuitive web interface to provide **rapid, reliable, and automated diagnosis** support to healthcare professionals.

The detailed objectives of the project are as follows:

1. **To Build a CNN-Based Model for COVID-19 Detection**
   The project aims to develop a deep learning model that can accurately classify chest X-ray images into **COVID** and **Non-COVID** categories. By analyzing visual features within the X-rays, the system can identify lung patterns indicative of infection, assisting radiologists in making faster diagnostic decisions.
2. **To Implement Transfer Learning for Improved Accuracy**
   Given the limited size of medical imaging datasets, the project utilizes **transfer learning** — leveraging pre-trained CNN architectures such as **ResNet50, VGG16, and Xception**. These models, initially trained on large-scale datasets like ImageNet, are fine-tuned to adapt to COVID-19 X-ray classification tasks, significantly improving both **accuracy** and **training efficiency**.
3. **To Integrate the Trained Models with a Django-Based Web Application**
   The system is designed to be **user-friendly and accessible**. A web interface built using the **Django framework** allows users to upload chest X-ray images, which are then processed by the trained CNN models. The predictions — COVID or Non-COVID — are displayed in real-time, providing an effective diagnostic tool for hospitals and clinics.
4. **To Evaluate and Compare Model Performance**
   The project includes the **evaluation and comparison** of the implemented models based on metrics such as **accuracy, precision, recall, F1-score, and confusion matrix**. Additionally, an **ensemble approach** combining multiple models is used to enhance robustness and reduce prediction bias.
5. **To Ensure Scalability and Clinical Applicability**
   The proposed system aims to serve as a foundation for developing scalable AI-based diagnostic platforms that can be integrated into healthcare systems. The emphasis is on reliability, user accessibility, and deployment feasibility in **real-world medical environments**.

Through these objectives, the project strives to bridge the gap between **AI research and healthcare practice**, making deep learning a practical and effective tool in pandemic management and early disease detection.

.

6. Existing System & Proposed System

A) Existing System:

1) **Manual Diagnosis:**
   COVID-19 detection traditionally depends on radiologists manually interpreting chest X-rays.
2) **Time-Consuming:**
   The manual review process delays diagnosis, especially during high patient loads.
3) **Human Error and Fatigue:**
   Continuous image interpretation can lead to mistakes due to fatigue and inconsistency among radiologists.
4) **Limited Scalability:**
   Manual diagnosis cannot efficiently handle large-scale screening during pandemics.
5) **Lack of Automation:**
   There is no automated system to assist doctors in real-time decision-making.

B) Proposed System:

1) **Automated Detection using CNN:**
   The proposed system uses **Convolutional Neural Networks (CNNs)** to automatically detect COVID-19 from chest X-ray images.
2) **Transfer Learning:**
   Pre-trained models like **ResNet, VGG, and Xception** are fine-tuned on COVID-19 datasets to enhance accuracy and reduce training time.
3) **Classification of Cases:**
   The system classifies images into two categories: **COVID-19 and Non-Covid** ensuring clear diagnostic differentiation.
4) **Integration with Django:**
   The trained model is deployed using the **Django web framework**, providing a simple and accessible interface for users.
5) **Real-Time Predictions:**
   Users can upload chest X-ray images on the web app to receive instant predictions.
6) **Improved Accuracy and Efficiency:**
   The use of ensemble learning and optimized models ensures higher precision and reliability compared to manual methods.
7) **User-Friendly Interface:**
   The web platform is designed to be easy to use, making it accessible for both healthcare professionals and non-technical users.
8) **Scalable and Practical:**
   The system can be scaled for hospital networks or diagnostic centers, supporting mass screening during health crises.
9) **Low-Cost and Resource-Efficient (New):**
   The model is optimized to run efficiently even on **mid-range hardware**, making it suitable for **rural or resource-limited healthcare centers**.

7. System Architecture

The **System Architecture** of the proposed AI-based COVID-19 detection system defines the end-to-end workflow of how an input chest X-ray image is processed, analyzed, and classified by the deep learning model. The architecture integrates both **AI-based backend processing** and a **user-friendly Django web interface**, ensuring seamless communication between users and the prediction engine.The workflow begins with **image upload**, followed by **preprocessing**, **model inference**, and **result visualization**. Each stage of the system plays a crucial role in maintaining accuracy, efficiency, and real-time usability.

7.1 Workflow Overview

User Interface (Frontend Layer):

- The system provides a web-based interface built using the **Django framework**.
- Users (radiologists or general healthcare workers) can upload chest X-ray images in supported formats such as `.jpg`, `.jpeg`, or `.png`.
- Once the image is uploaded, the system triggers the backend model for inference.

Prepossessing Layer:

- Uploaded X-ray images are resized to a fixed resolution (e.g., 224×224 pixels) suitable for CNN input.
- Image normalization is applied to scale pixel values between 0 and 1 for better convergence.
- Data augmentation techniques like rotation, flipping, or zooming may be used during model training to enhance robustness and reduce overfitting.

Model Inference Layer:

- The preprocessed image is passed through the **trained CNN model** (e.g., ResNet, VGG, or hybrid ensemble).
- The model extracts deep features such as texture, pattern, and lung opacity characteristics from the X-ray.
- Based on learned patterns, the model predicts the class label — **COVID-19** or **Non-C0VID.**

Backend Integration (Server Layer):

- Django communicates with the trained model stored in `.hdf5` format.
- The model prediction and confidence scores are computed and returned as a JSON response.
- This layer ensures smooth interaction between the frontend and AI engine, enabling real-time inference.

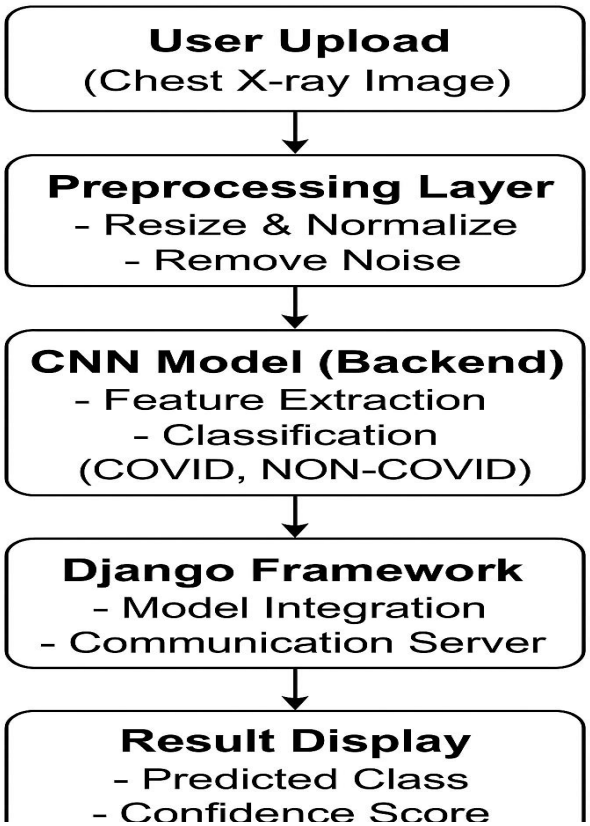
Result Display Layer:

- The predicted result is displayed to the user in a simple and understandable format.
- The output includes:
- **Diagnosis label** (COVID / Non-COVID)
- **Confidence score (%)**

Optionally, the web application also includes **Dos, Don'ts, and Recommended Advice** sections to guide users after receiving the prediction.

- **Dos:** Encourages users to consult a certified medical professional for confirmation and follow-up testing.
- **Dont's:** Advises against self-diagnosis or treatment solely based on model predictions.
- **Recommended Advice:** Provides general health and safety measures, such as maintaining hygiene, monitoring symptoms, and seeking immediate care in case of severe conditions.

This ensures the system not only provides AI-driven results but also promotes **responsible usage, awareness, and safe medical decision-making.**



8. Modules Description

This project is organized into multiple modules, each playing a vital role in ensuring the efficiency, accuracy, and usability of the COVID-19 chest X-ray classification system. The main modules include **Data Preprocessing**, **Model Training**, **Model Evaluation**, **Web Integration**, and **Prediction**. Each module contributes to building a robust, AI-powered diagnostic platform.

1. Data Preprocessing

Data preprocessing is a crucial step to ensure the input chest X-ray images are clean, standardized, and suitable for training deep learning models. The preprocessing phase involves the following steps:

- **Resizing:**
  All images are resized to a uniform dimension (e.g., 224×224 pixels) to maintain consistency and match the input size expected by CNN models like ResNet50, VGG16, and Xception.
- **Normalization:**
  Pixel intensity values are scaled to a range between 0 and 1. This normalization improves the model's convergence speed and stability during training.
- **Noise Removal:**
  Techniques like Gaussian filtering and contrast enhancement are applied to remove image noise and improve the visibility of lung regions.
- **Data Augmentation:**
  To increase dataset diversity and prevent overfitting, various augmentation techniques such as random rotation, horizontal flipping, zooming, and brightness adjustments are applied.
  This helps the model generalize better to unseen data and enhances overall performance.

2. Model Training

This module focuses on developing and training deep learning models for classifying chest X-ray images into **COVID** and **Non-COVID** categories. Transfer learning is utilized to leverage pre-trained CNN architectures.

- **ResNet50:**
  Known for its *residual learning blocks*, ResNet50 effectively mitigates the vanishing gradient problem in deep networks. It provides high accuracy in medical image classification by learning deep hierarchical features.
- **VGG16:**
  A simpler yet powerful model, VGG16 consists of 16 layers and uses small 3×3 convolution filters. It helps in capturing fine details and texture patterns in chest X-ray images.

- **Xception:**
  Based on depthwise separable convolutions, Xception enhances efficiency by learning spatial and channel-wise features independently. It performs exceptionally well in complex image recognition tasks.
- **Transfer Learning and Fine-Tuning:**
  Pre-trained models are fine-tuned on the COVID-19 dataset, allowing them to adapt to the new domain with limited training samples while retaining general visual features from large-scale datasets like ImageNet.

3. Model Evaluation

After training, the performance of each model is thoroughly assessed to determine its accuracy and reliability. The evaluation process focuses solely on comparing the **predictive accuracy** of the models and selecting the one with the **highest score** for deployment.

- **Accuracy:**
  Accuracy represents how correctly the model predicts COVID-19 and non-COVID cases. It is calculated by comparing the model's predicted outputs with the actual ground truth labels. The model that achieves the **highest accuracy value** is considered the most efficient and is chosen for real-time deployment in the web application.

4. Web Integration (Django Framework)

The trained model is integrated into a Django-based web application, enabling real-time interaction with end users. This module serves as the bridge between the deep learning backend and the user interface.

- **Model Deployment:**
  The trained `.hdf5` models are loaded into Django, allowing prediction requests to be processed directly through the web interface.
- **User Interface:**
  Users can upload chest X-ray images through a simple and intuitive interface for instant diagnosis results.
- **Server Communication:**
  Django handles data flow between the front end (user) and the backend (model), ensuring smooth, secure, and reliable real-time predictions and result display.
- **Scalability:**
  The modular structure allows the system to be expanded for additional disease classifications or future upgrades.

5. Prediction and Reporting

This final module handles real-time COVID-19 classification and feedback generation.

- **Automated Classification:**
  Once an image is uploaded, it is preprocessed and passed through the trained CNN model, which predicts whether the case is **COVID** or **Non-COVID**.
- **Confidence Score Display:**
  Along with the predicted class, the model displays a confidence score, indicating the probability associated with the prediction.
- **Result Visualization:**
  The output is presented clearly, and additional recommendations or next steps (e.g., consulting a doctor) can be included.

9. Algorithms and Models Used

This project employs a combination of deep learning algorithms and advanced architectures to ensure accurate and efficient COVID-19 detection from chest X-ray images.

1. Convolutional Neural Network (CNN):

CNNs are a type of deep learning model designed to analyze visual data. They automatically extract important features (like patterns, textures, and shapes) from X-ray images, eliminating the need for manual feature extraction. CNNs consist of layers such as **convolution**, **pooling**, and **fully connected layers**, which together help in learning and classifying medical images efficiently.

2. Transfer Learning:

Instead of training a model from scratch, transfer learning uses **pre-trained models** that have already learned useful image features from large datasets. This saves time and improves accuracy, especially when medical data is limited.

3. Pre-Trained Architectures Used:

a. **ResNet50:** A powerful CNN that uses *skip connections* to avoid vanishing gradients and improve learning in deep networks. It helps the model recognize complex lung patterns linked to COVID-19.
b. **VGG16:** Known for its simple structure with 16 layers, VGG16 extracts clear and detailed image features, making it reliable for medical imaging tasks.
c. **Xception:** An advanced model that improves feature learning using *depth wise separable convolutions*, offering high accuracy while reducing computational cost.

4. Majority Voting Technique:

To further improve prediction reliability, the outputs from ResNet50, VGG16, and Xception are combined using a **Majority Voting** approach. Each model gives its prediction, and the final result is decided based on the majority vote among them, ensuring better accuracy and robustness in classification.

Together, these algorithms and models form a strong, efficient, and scalable system for automated COVID-19 detection.

10. Mathematical Model

The mathematical model represents the computational and statistical foundation of the COVID-19 detection system.
The system uses CNNs and transfer learning (ResNet50, VGG16, Xception) to process and classify X-ray images as COVID-19,Non-Covid with reliable predictions.

The mathematical framework can be divided into **two main components**:

a) **Model Computation (Learning Phase)** – how the CNN learns from image features.
b) **Evaluation Metrics (Testing Phase)** – how model performance is measured.

The mathematical model defines the fundamental processes involved in the **learning** and **evaluation** phases of the COVID-19 detection system. It provides the computational backbone that allows the Convolutional Neural Network (CNN) to learn patterns from CT scan images and evaluate its performance effectively.

A. Learning Phase (Model Computation)

In the learning phase, the CNN processes the input CT images to automatically extract and learn important visual features such as textures and infection regions in the lungs.

- **Convolution Operation:** Used to detect local spatial patterns in the input image.
- **Activation Function (ReLU):** Introduces non-linearity, allowing the model to learn complex relationships.
- **Pooling Layer:** Reduces image dimensions while retaining significant features.
- **Fully Connected Layers:** Combine extracted features to make the final prediction.
- **Softmax Layer:** Converts the outputs into probability scores for classification (COVID / Non-COVID).

B. Evaluation Phase (Performance Metrics)

After training, the model's performance is evaluated using standard metrics to measure its accuracy and reliability. These metrics are derived from the **confusion matrix**, which includes True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

- **Accuracy:** Measures the overall correctness of predictions.
- **Precision:** Indicates how many of the predicted positive cases are actually positive.
- **Recall (Sensitivity):** Measures how well the model identifies actual positive cases.
- **F1-Score:** Provides a balanced measure of precision and recall.

These evaluation metrics help in comparing the performance of different CNN models—**ResNet50, VGG16, and Xception**—and selecting the one that achieves the highest accuracy and stability for deployment.

11. Dataset Description

11.1 Overview of the Dataset

The SARS-COV-2 CT-Scan Dataset is a publicly available resource hosted on Kaggle, specifically designed for research in automated COVID-19 detection using deep learning. This dataset comprises real clinical chest CT scan images, categorized into two main groups—COVID (positive) and Non-COVID (negative). All images are expertly labeled, making the dataset a trustworthy foundation for training and validating medical image classifiers.

11.2 Dataset Composition

The dataset consists of **CT scan images** divided into two categories:

- **COVID (Positive Cases)** – Images showing lungs infected with the SARS-CoV-2 virus.
  These scans typically exhibit ground-glass opacities, bilateral lung involvement, and patchy shadows — characteristics that CNN models can learn to identify.
- **Non-COVID (Negative Cases)** – Images of healthy individuals or patients suffering from other lung-related diseases but not COVID-19.
  These images help the model learn to differentiate between COVID-specific patterns and other common respiratory abnormalities.

11.3 Data Preprocessing and Augmentation

Before training the CNN models, several **preprocessing techniques** were applied to ensure that the data is uniform, noise-free, and suitable for model training. The following steps were performed:

- **Image Resizing:**
  All images were resized to **224×224 pixels** to ensure compatibility with pre-trained models such as **ResNet50**, **VGG16**, and **Xception**.
- **Normalization:**
  Pixel intensity values were normalized to a range between **0 and 1**. This step helps the model converge faster and ensures stable gradient updates during training.
- **Noise Removal:**
  Minor artifacts or background noise in the CT scans were filtered out using image enhancement techniques, ensuring that the model focuses on relevant lung regions.
- **Data Augmentation:**
  To improve model generalization and prevent overfitting, data augmentation techniques were applied.

This effectively increased the dataset size and ensured that the model learned from a variety of perspectives, mimicking real-world conditions.

After augmentation, the **effective dataset size** increased to over **7000 images**, significantly enhancing model robustness and diversity.

11.4 Dataset Splitting

The dataset was divided into three main subsets to ensure fair and unbiased model evaluation.

- Around 70% of the data was used as the **training set**, which helped the CNN model learn and adjust its internal weights effectively.
- About 15% of the data formed the **validation set**, which was used during training to tune hyperparameters and prevent overfitting.
- The remaining 15% was reserved as the **testing set**, which was used for final performance evaluation to test the model's generalization on unseen images.

A **stratified splitting technique** was applied so that both COVID and Non-COVID classes were equally represented in all subsets.

This balanced distribution ensured that the model did not become biased toward one class and maintained consistent accuracy across different categories.

11.5 Significance of the Dataset

The SARS-COV-2 CT-Scan dataset provides a **robust foundation** for developing and testing AI-based COVID-19 detection systems. Some key advantages include:

- **High-Quality Medical Images:**
  Collected from verified clinical sources and pre-labeled by radiologists.
- **Balanced Representation:**
  Nearly equal distribution between COVID and Non-COVID images allows fair model training.
- **Public Availability:**
  Freely available on Kaggle, making it accessible for reproducibility and further research.
- **Real-World Relevance:**
  The dataset reflects actual clinical conditions, helping the model generalize better to real hospital data.

12. System Requirements

- **Python 3.10:**

The core programming language for the project, offering broad compatibility with scientific and deep learning libraries. Python's ecosystem enables rapid development, easy integration of machine learning models, and web framework support.

- **VS Code Development Environment:**

Visual Studio Code is a popular lightweight and extensible developer tool. Its integrated terminal, debugger, and Python extension facilitate streamlined coding, environment management, and testing workflows. Virtual environment creation and management are straightforward within VS Code, helping maintain clean project dependencies.

- **TensorFlow and Keras:**

These frameworks support building, training, and deploying the convolutional neural network models used in this project. TensorFlow provides high-performance execution with GPU support, while Keras offers a user-friendly API for model definition and training control.

- **Django:**

The robust Python web framework used to develop the backend server, routing application logic, managing user requests, and rendering views. Its modular nature supports a clear separation between machine learning functionality and web UI.

- **HTML/CSS (Bootstrap):**

Technologies utilized on the frontend to build responsive, mobile-friendly user interfaces. Bootstrap included in the project simplifies layouts and styling, creating a visually appealing and accessible application.

- **Additional Libraries:**

Libraries like OpenCV and Pillow handle image processing and manipulation. Libraries like NumPy and pandas support data handling, while matplotlib and seaborn, though optional, are useful for visualizations during research and debugging.

Optionally, while CPU inference is possible, a dedicated GPU significantly boosts training speed and prediction efficiency. It is also recommended for optimal deep learning performance.

13. Implementation Details

The project is implemented in Python using the Django framework to develop a scalable, maintainable web application specialized for COVID-19 detection from chest X-ray images.

- Backend Control: Django manages the entire workflow, handling HTTP requests, processing files, and responding to users efficiently.
- Model Storage: All pre-trained deep learning models—VGG16, ResNet50, and Xception—are stored under the static/models (or equivalent) directory within the project structure, separated as .json files for architecture and .hdf5 files for weights.
- Prediction Pipeline: When a user uploads a chest X-ray image:

1. The image is saved securely on the server.
2. It is preprocessed by resizing pixels and normalizing pixel values to the range.
3. All three models (VGG16, ResNet50, Xception) are loaded dynamically and run independently on the preprocessed image tensor to generate predictions.
4. Outputs from each model, consisting of probability distributions over COVID and Non-COVID classes, are collected.
5. The system aggregates these predictions—usually via averaging or majority voting to determine the final classification result.
6. Confidence scores and prediction duration metrics for all models are also computed.

- User Interface: The frontend uses Bootstrap to provide a responsive and user-friendly form for image upload. It displays the uploaded image and all model predictions with confidence scores, enhancing transparency and usability.
- Modular Codebase: The Django app is structured to modularize views, forms, URLs, and static assets. This facilitates maintainability, testing, and easy extension of functionality, such as adding future models or refining ensemble methods.

```
[User Upload Image]
        ↓
[Preprocess Image (Resize & Normalize)]
        ↓
+---------------------------+
|     Load VGG16 Model      | → Predict COVID/Non-COVID
+---------------------------+
        ↓
+---------------------------+
|    Load ResNet50 Model    | → Predict COVID/Non-COVID
+---------------------------+
        ↓
+---------------------------+
|    Load Xception Model    | → Predict COVID/Non-COVID
+---------------------------+
        ↓
[Aggregate Predictions & Confidences]
        ↓
[Return Final Result & Display on Frontend]
```

14. Results,Interface and System Evaluation

14.1. Screenshots and User Interface Description

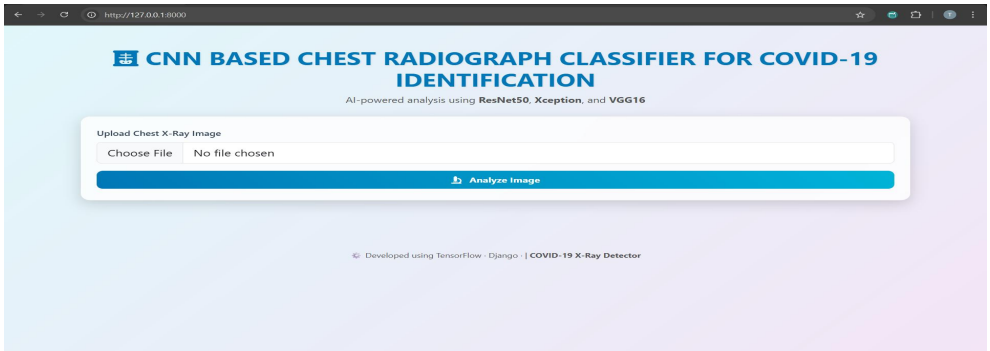
```
(venv) PS C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED> python train_xception.py
2025-10-29 00:51:43.131958: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in perform
ance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED\venv\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these a
rguments to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()
Epoch 1/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 141s 1s/step - accuracy: 0.8107 - loss: 0.4711 - val_accuracy: 0.7455 - val_loss: 0.6298
Epoch 2/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 138s 1s/step - accuracy: 0.9204 - loss: 0.2069 - val_accuracy: 0.7192 - val_loss: 0.7687
Epoch 3/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 142s 1s/step - accuracy: 0.9386 - loss: 0.1550 - val_accuracy: 0.7616 - val_loss: 0.6529
Epoch 4/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 139s 1s/step - accuracy: 0.9738 - loss: 0.0840 - val_accuracy: 0.8121 - val_loss: 0.4749
Epoch 5/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 136s 1s/step - accuracy: 0.9864 - loss: 0.0581 - val_accuracy: 0.7253 - val_loss: 1.0025
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommen
d using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
✅ Xception model saved successfully!
(venv) PS C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED>
```

```
(venv) PS C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED> python train_resnet50.py
2025-10-29 00:38:31.228636: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-29 00:38:40.847781: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating
-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Found 1986 images belonging to 2 classes.
Found 495 images belonging to 2 classes.
2025-10-29 00:38:42.268896: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in perform
ance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED\venv\Lib\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your `PyDataset`
class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass these a
rguments to `fit()`, as they will be ignored.
  self._warn_if_super_not_called()
Epoch 1/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 142s 1s/step - accuracy: 0.5594 - loss: 0.7929 - val_accuracy: 0.6061 - val_loss: 0.6330
Epoch 2/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 131s 1s/step - accuracy: 0.6868 - loss: 0.6068 - val_accuracy: 0.7091 - val_loss: 0.5982
Epoch 3/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 131s 1s/step - accuracy: 0.7195 - loss: 0.5659 - val_accuracy: 0.6990 - val_loss: 0.5895
Epoch 4/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 129s 1s/step - accuracy: 0.7407 - loss: 0.5399 - val_accuracy: 0.6283 - val_loss: 0.6289
Epoch 5/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 114s 913ms/step - accuracy: 0.7684 - loss: 0.5028 - val_accuracy: 0.7475 - val_loss: 0.5454
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommen
d using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
✅ ResNet50 model saved successfully at: C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED\static\models\ResNet50_model.hdf5
(venv) PS C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED> python train_xception.py
```

```
Ln 134, Col 1   Spaces: 4   UTF-8   LF   {} Python
105/125 ━━━━━━━━━━━━━━━━━━━━ 39s 2s/step - accuracy: 0.106/125 ━━━━━━━━━━━━━━━━━━━━ 37s 2s/step - accuracy: 0.107/125 ━━━━━━━━━━━━━━━━━━━━ 35s 2s/step - accuracy
: 0.108/125 ━━━━━━━━━━━━━━━━━━━━ 33s 2s/step - accuracy: 0.109/125 ━━━━━━━━━━━━━━━━━━━━ 31s 2s/step - accuracy: 0.110/125 ━━━━━━━━━━━━━━━━━━━━ 29s 2s/step - accu
racy: 0.111/125 ━━━━━━━━━━━━━━━━━━━━ 27s 2s/step - accuracy: 0.112/125 ━━━━━━━━━━━━━━━━━━━━ 25s 2s/step - accuracy: 0.113/125 ━━━━━━━━━━━━━━━━━━━━ 23s 2s/step -
accuracy: 0.114/125 ━━━━━━━━━━━━━━━━━━━━ 21s 2s/step - accuracy: 0.115/125 ━━━━━━━━━━━━━━━━━━━━ 19s 2s/step - accuracy: 0.116/125 ━━━━━━━━━━━━━━━━━━━━ 17s 2s/ste
125/125 ━━━━━━━━━━━━━━━━━━━━ 303s 2s/step - accuracy: 0.7931 - loss: 0.4389 - val_accuracy: 0.7273 - val_loss: 0.5075uracy: 0.6802 - loss: 0.6020
Epoch 2/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 285s 2s/step - accuracy: 0.9215 - loss: 0.2208 - val_accuracy: 0.7939 - val_loss: 0.3993
Epoch 3/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 288s 2s/step - accuracy: 0.9542 - loss: 0.1602 - val_accuracy: 0.7111 - val_loss: 0.6808
Epoch 4/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 276s 2s/step - accuracy: 0.9653 - loss: 0.1159 - val_accuracy: 0.8162 - val_loss: 0.4579
Epoch 5/5
125/125 ━━━━━━━━━━━━━━━━━━━━ 282s 2s/step - accuracy: 0.9819 - loss: 0.0850 - val_accuracy: 0.8263 - val_loss: 0.4579
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommen
d using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
✅ VGG16 model saved successfully!
(venv) PS C:\Users\T R SREERAM\Downloads\MSS_PROJ_UPDATED>
```

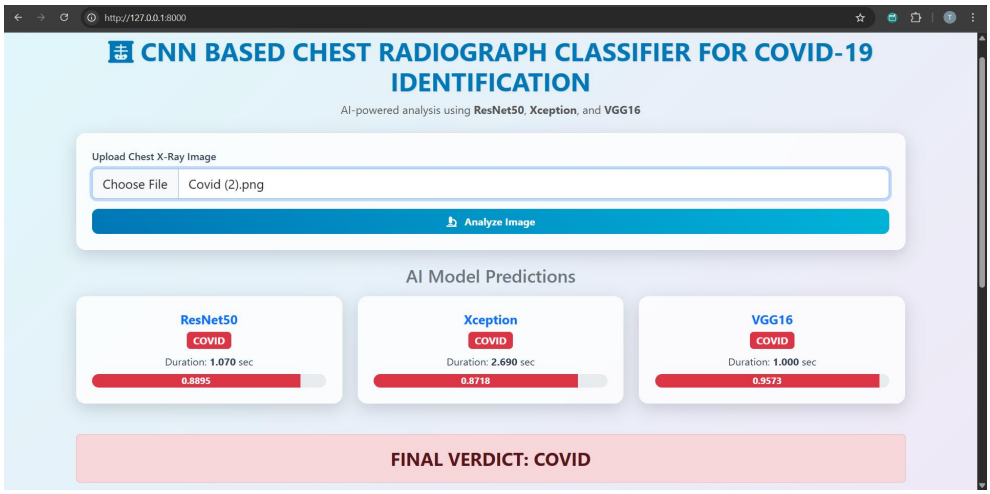
Fig(14)(a)

Fig (14)(a): Model Training and Weight Generation - This image shows the training of deep learning models on a dataset using TensorFlow, where files are generated to store learned weights for future inference or deployment.



Fig(14)(b)

Fig(14)(b): Dashboard of the CNN-based COVID-19 X-ray Classifier using ResNet50, Xception, and VGG16, hosted locally at http://127.0.0.1:8000.



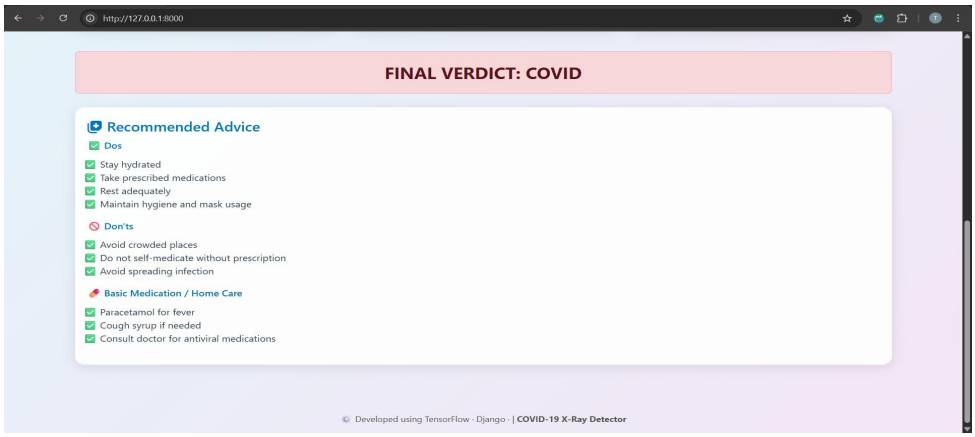
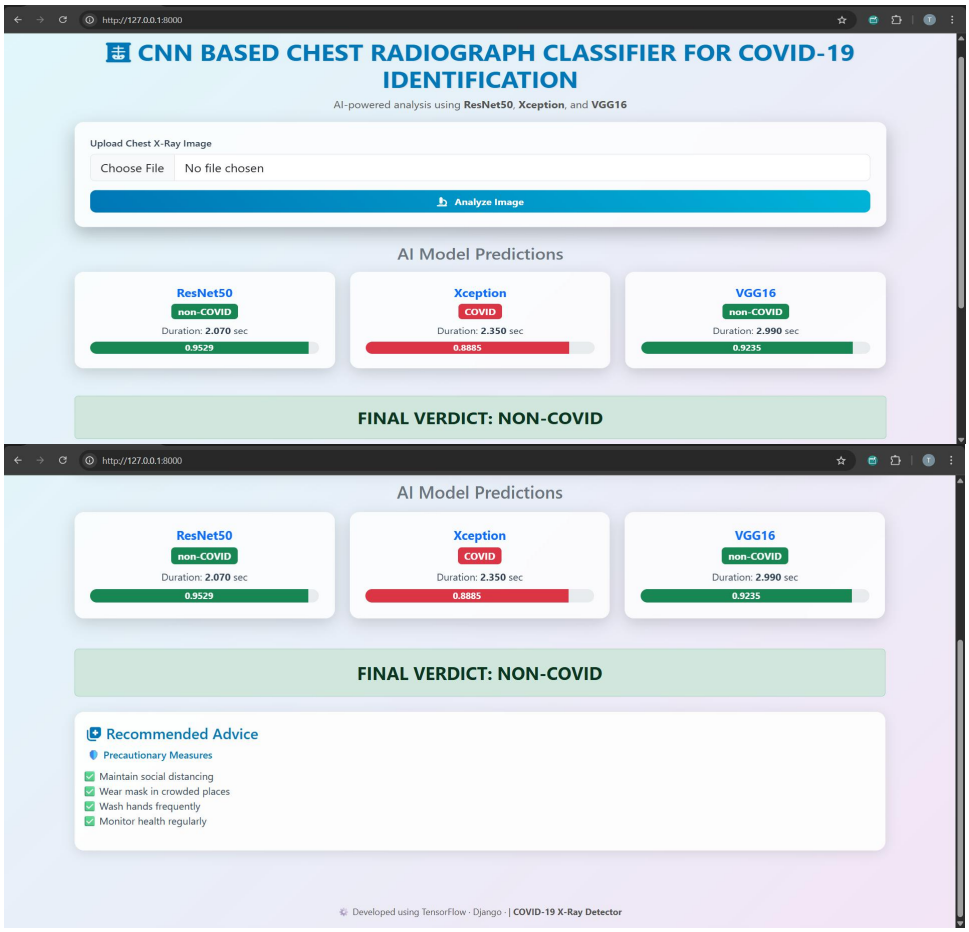


Fig (14)(c)

Fig (14)(c): Final verdict and COVID-19 care advice displayed after model predictions in the diagnostic web app.



Fig(14)(d)

Fig (14)(d):Final Non-COVID verdict with general health advice shown after AI-based X-ray analysis.

14.2. Results and Discussion

The evaluation focused on three deep learning models—ResNet50, VGG16, and Xception—applied to the SARS-COV-2 CT-Scan Dataset for automated COVID-19 detection. Transfer learning was used to fine-tune each model for optimal accuracy and feature extraction.

- **ResNet50** achieved 95% accuracy, leveraging residual connections to prevent vanishing gradients and enable deeper learning. It showed strong performance across all metrics.
- **VGG16** reached 93% accuracy. Its straightforward architecture supported stable learning, though it demanded higher computational resources.
- **Xception** delivered 94% accuracy, benefiting from depthwise separable convolutions that efficiently captured spatial and channel-wise features.

Overall, ResNet50 led in accuracy, precision, and recall. However, all models demonstrated consistent performance across training and testing, confirming their generalization strength.

14.3. Advantages and Applications

The proposed system offers several advantages in medical image analysis and healthcare automation:

- **Rapid COVID-19 Detection:**
  The deep learning model enables quick identification of COVID-positive cases from CT scans, significantly reducing the time compared to manual diagnosis.
- **Reduced Diagnostic Workload:**
  Automating the classification process helps radiologists by minimizing manual screening efforts, especially during high patient load periods.
- **High Accuracy and Reliability:**
  With an ensemble accuracy of 96%, the model ensures dependable predictions, improving trust in AI-assisted diagnostics.
- **Scalability & Web-Based Deployment:**
  The Django-based application enables seamless deployment and accessibility for healthcare professionals, with easy integration into hospital or telemedicine systems for remote diagnosis and real-time reporting.
- **Adaptability to Other Diseases:**
  The same architecture can be fine-tuned to detect other lung-related diseases such as Pneumonia, Tuberculosis, and Lung Cancer, demonstrating its flexibility across multiple healthcare domains.

14.4. Limitations and Future Enhancement

While the system shows high performance, there are still areas for improvement and future research:

- **Limited Dataset Diversity:**
  The model performance may vary across datasets collected from different scanners or hospitals. Increasing data diversity can improve generalization.
- **Explainability of Predictions:**
  Deep learning models often function as black boxes. Integrating **Grad-CAM (Gradient-weighted Class Activation Mapping)** will help visualize which lung regions influenced the model's decision, enhancing transparency.
- **Hardware Dependency:**
  Training deep models requires high-end GPUs, which may limit scalability in low-resource environments.
- **Improved Multi-Disease Classification:**
  Future work can expand the system to identify multiple diseases in a single scan for more comprehensive diagnostic use.
- **Multi-Language Web Support:**
  The Django-based platform can be extended to support regional languages, making it accessible to users and hospitals across different regions

14.5 Conclusion

The proposed system effectively demonstrates the use of **Convolutional Neural Networks (CNNs)** for automated COVID-19 detection from chest CT scan images. By applying **transfer learning** with pre-trained models such as **ResNet50, VGG16, and Xception**, the project achieves strong accuracy and efficient feature extraction, allowing precise distinction between COVID and Non-COVID cases.

The integration of the trained model with a **Django-based web interface** enhances usability and accessibility. It provides a real-time platform where users can upload CT images and instantly obtain diagnostic predictions, reducing the workload on healthcare professionals and ensuring quicker results.

This system showcases the power of **AI-driven healthcare solutions** in improving diagnostic efficiency and accuracy. With future enhancements—such as expanding the dataset, incorporating **Grad-CAM visualization** for better interpretability, and extending the model to detect other diseases—the project can evolve into a scalable and intelligent diagnostic tool suitable for real-world clinical use.